GlassML: Panel-Level Glass Interposer with Reconfigurable Optical Fabric For LLM Training

HPCA 2027 Submission **#1373**
Confidential Draft
Do NOT Distribute!!

***Abstract*—Distributed training of large-scale LLM relies heavily on the performance of the scale-up interconnect. Existing scale-up architectures face two limits: wafer-scale integration is bounded by stagnating wafer dimensions, whereas scale-up rack systems incur high costs for off-package links and switches. Panel-level glass interposers with embedded optical waveguides offer a promising alternative by significantly expanding the in-package footprint and routing density, allowing scale-up traffic to remain strictly on-substrate. Yet, system architectures that enable panel-level glass interposers to serve as an LLM scale-up domain remain unexplored.**

We present GlassML, an architecture and system design on a panel-level glass interposer with an embedded reconfigurable optical fabric for LLM training. At the architecture level, GlassML integrates inner, edge, and controller tiles to support computation, communication, and panel-wide control and coordination. At the switching level, GlassML introduces a reconfigurable, high-bandwidth, and scalable multi-plane optical fabric, together with a waveguide ownership model that reduces the required switching elements by $8\times$ while providing structured, conflict-free lightpaths across the panel. At the scheduling level, GlassML formulates collective communication as a routing and scheduling problem over demand matrices, supporting both precomputed regular collectives and runtime-dependent AllToAll communication in expert-parallel MoE training. Under measured MoE routing skew, GlassML accelerates AllToAll by up to 8.58$\times$ over Waferscale and 5.84$\times$ over NVL64. Across nine training workloads, GlassML improves training throughput by up to 40.2% and 26.1% over Waferscale and NVL64, respectively.

I. Introduction

Large language models (LLMs) have grown rapidly in scale in recent years. The increasing number of model parameters has driven corresponding growth in computational requirements, memory footprint, and network demands. State-of-the-art LLMs have entered the trillion-parameter regime, requiring multiple TBs of accelerator memory to store model weights alone [3], [20], [64], [80]. Meanwhile, accelerator compute dies have reached the silicon lithography reticle limit of 858 mm^2 [16], [17], which is the maximum rectangular area that can be exposed in a single lithography step, leaving no room to increase compute capability through larger dies.

Consequently, large-scale LLM training has become fundamentally distributed: model parameters, optimizer states, and activations must be partitioned across multiple accelerators to fit in device memory [21], [25], [31], [58], [69]. For example, Context Parallelism (CP) partitions the sequence dimension of KV tensors, whereas Expert Parallelism (EP) shards the expert weights of an MoE model across accelerators. These parallelism approaches introduce expensive collective communication across distributed ranks, making interconnect performance critical. Production measurements of MoE training report the two AllToAll phases of expert parallelism occupying 33% to 68% of the training iteration time [43]. Modern LLM training systems organize GPUs into tightly coupled scale-up domains, where GPUs are connected by high-bandwidth, low-latency fabrics [32], [63], and employs communication-heavy model sharding within each domain to improve training throughput [43], [58]. As a result, the size, bandwidth, and cost of the scale-up domain directly determine which model-sharding strategies can be supported efficiently and, ultimately, the scalability of LLM training.

Existing systems construct large scale-up domains through two broad approaches: wafer-scale integration (WSI) and rack-scale switched fabrics. WSI uses an entire silicon wafer to integrate compute, memory, and network components. This approach attains high in-package signaling density through short-reach interconnects, including UCIe-class die-to-die links in interposer-based designs [45]. WSI can take either of two forms: monolithic wafer-scale processors, in which processing elements (PEs) and networks-on-chip (NoCs) are directly fabricated across the silicon wafer, such as the Cerebras WSE [44], [88]; or wafer-scale interposer systems that integrate separately tested known-good dies (KGDs) on a common silicon substrate [75], [78], [79], [89]. However, both forms of WSI remain bounded by the physical dimensions of silicon wafers, which have stagnated at 300 mm for decades [49].

In contrast, rack-scale systems construct a large scale-up domain by connecting discrete accelerators through a switched high-bandwidth fabric. For example, NVIDIA NVL72 connects 72 GPUs through a rack-scale non-blocking NVLink fabric [63]. However, at rack-scale distances, electrical channels incur higher loss and require more complex, slower signaling [53], [54], [66]. Replacing the long-reach electrical links with optical ones alleviates the signaling-rate vs. reach trade-off, yet the aggregate fabric bandwidth must still be handled by discrete switch dies whose radix and I/O capacity are bounded by the finite beachfront of reticle-limited switch dies [45], [63].

Panel-level glass interposers can overcome both scaling limits. They provide an integration area beyond silicon-wafer dimensions, and embed optical waveguides that sustain high bandwidth density over meter-scale on-substrate paths, forming a reconfigurable fabric through distributed switches that

are not limited by die beachfront. Recent advances in panel-level packaging (PLP) enable glass substrates as large as 600×600 mm, substantially larger than the area provided by 12-inch wafers [1], [8], [9], [18], [30], [60]. Glass also provides high dimensional stability, reduced warpage, fine-pitch redistribution layers (RDLs), and through-glass vias (TGVs) for signal and power delivery [15], [33], [83]. Fig. 1 compares this form factor with wafer-scale integration. Prior work has demonstrated low-loss optical waveguides embedded in large glass substrates [7], [11]–[13], [23], [71], [73]. Recent work opens a promising new point in the scale-up design space by proposing a panel-level glass interposer with an embedded reconfigurable optical fabric [30], but does not address the critical architectural, algorithmic and system-level challenges to enable large LLM training on such systems.

Taking this opportunity, we present GlassML, the first architecture and system design to turn a panel-level glass interposer with an embedded reconfigurable optical fabric into a workload-ready scale-up domain for LLM training. Assembling tiles of state-of-the-art GPUs on a single panel, GlassML achieves the bandwidth density of die-to-die interfaces across the distances that today's systems reach only through rack-scale cabling: it delivers the connectivity of a rack-scale fabric with per-accelerator bandwidth attainable in wafer-scale integration, therefore significantly benefiting network-bound components in LLM training. Realizing this vision requires solving two fundamental challenges imposed by the embedded circuit-switched optical fabric.

Challenge 1. GlassML must provide high-bandwidth network across panel-scale distance without incurring prohibitive waveguide and switch complexity. Adding parallel waveguides increases both routing density and the number of optical crosspoints. Unlike datacenter optical fabrics, whose fiber-connected switches largely decouple topology from physical placement, GlassML embeds its waveguides and switches within a limited-space glass substrate. Its logical topology, physical layout, and switch organization must therefore be carefully designed.

Challenge 2. GlassML must efficiently execute LLM collectives over a circuit-switched optical fabric with reconfiguration costs, where every transfer requires a conflict-free lightpath to be configured before transmission. Distributed training presents both pre-determined communications patterns (TP, CP, DP, PP) as well as runtime-determined ones (EP with MoE). Therefore, this requires support of precomputed schedules and low-latency online scheduling for MoE while limiting fabric-reconfiguration overhead.

In this paper, we present the following contributions.

- We develop *GlassML*, the first architecture and system design that turns a panel-level glass interposer with an embedded reconfigurable optical fabric into an LLM training scale-up domain.
- To address Challenge 1 and achieve high system bandwidth without prohibitively increasing switching complexity and optical loss, we distribute the required waveguides over K independent planes rather than aggregating, which reduces the number of required switching elements and keeps en-route optical power loss independent of aggregate bandwidth.
- To address Challenges 1&2 and sustain arbitrary permutation traffic, such as MoE AllToAll traffic, on a bufferless optical circuit-switched fabric, we prove that n WGs per row and column are both necessary and sufficient and that XY routing is optimal (Theorem 1), and we introduce a waveguide ownership model that maps every permutation to a unique set of conflict-free lightpaths.
- To address Challenge 2 and execute both predetermined and runtime-determined collectives on reconfigurable optical fabric, we formulate them as demand matrices decomposed into weighted permutations across K planes by extending the polynomial-time SPECTRA [42] algorithm.
- We evaluate GlassML against wafer-scale and rack-scale baselines. Under measured MoE routing skew, GlassML accelerates AllToAll by up to $8.58\times$ over Waferscale and $5.84\times$ over NVL64, and improves end-to-end training throughput by up to 40.2% and 26.1%, respectively.

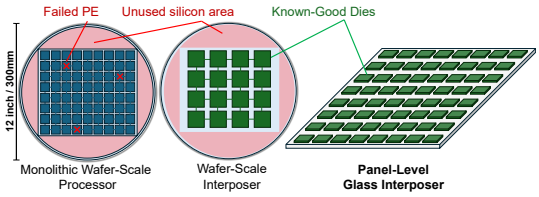


Fig. 1: Comparison of a Panel-Level GIP with two forms of WSI.

II. BACKGROUND

A. Mixture-of-Experts LLMs

The LLM architecture has shifted from dense monolithic models to Mixture-of-Experts (MoE) [2], [3], [20], [21], [34], [36], [57], [74]. Compared with dense Transformer models, Mixture-of-Experts (MoE) replaces the monolithic multi-layer perceptron (MLP) module in each Transformer block with a sparsely activated expert module, as shown in Fig. 2. Before the expert computation, a router computes an affinity score between each input token and each expert, and selects the top-k experts for every token. The token performs matrix multiplications with k small expert weights, before aggregating into output activation for the expert MLP module. Due to their sparse nature, MoE models scale better in training owing to the reduced forward pass cost compared to their dense counterparts [4], [37]. This in part fueled the emergence of trillion-parameter models such as Deepseek-V4 1.6T and Kimi-K3 [20], [80].

The growth in model sizes far exceeds the growth of GPU VRAM capacity. Consequently, large-scale LLM training uses multiple forms of parallelisms to shard the model and the training workload in order to reduce VRAM usage. We consider 4 common sharding strategies. Data Parallel (DP) shards the batch dimension B. Pipeline Parallel (PP) shards the layer dimension of the model L. Tensor Parallel (TP) [58]

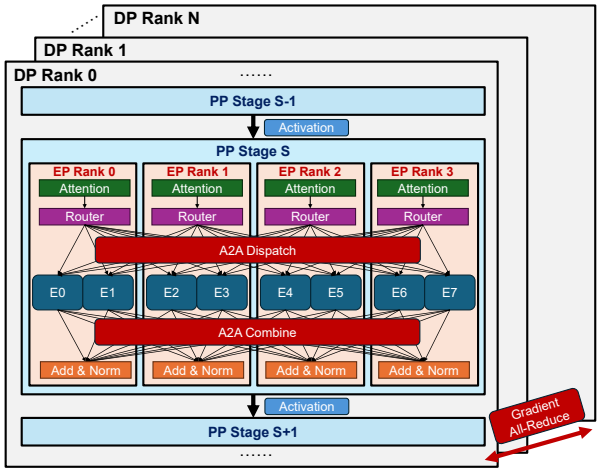


Fig. 2: Distributed training for MoE models using EP, DP, and PP.

shards the hidden dimension H and the intermediate dimension of the MLP modules. Expert Parallel (EP) [39] distributes individual experts to GPU ranks. Parallelism strategies trade communication for reduced computation and memory usage per GPU. DP requires gradient synchronization across DP replicas. TP requires collective communication within each transformer layer before and after the computation of the sharded weight tensor. PP introduces point-to-point activation and gradient transfers between adjacent pipeline stages. EP introduces AllToAll communication to dispatch tokens to their selected experts on different ranks and to aggregate the results of individual expert forward passes.

B. Optical Communication

Optical communication carries data by modulating light and propagating it through a waveguide or fiber, enabling high-bandwidth transmission over longer distances than electrical interconnects. At the source, a continuous-wave light source supplies optical power to a modulator in the photonic integrated circuit (PIC), while a transmitter (TX) in the electronic integrated circuit (EIC) provides the electrical signal from serialized data. The modulator maps the electrical signal to changes in optical power, producing a modulated optical signal that propagates to the destination. On the receiving end, a photodetector converts the received optical power into photocurrent, which is then converted and amplified by an electrical receiver (RX) to recover the data. The source and destination thus form an Electrical/Optical/Electrical (E/O/E) data path, while data remains in the optical domain during transmission without intermediate O/E/O conversion or repeaters [30], [53], [77].

The data rate of a single wavelength is limited by the bandwidth of its modulator and photodetector. Wavelength-division multiplexing (WDM) increases the bandwidth of a physical waveguide by allowing multiple independently modulated wavelengths to propagate simultaneously [14], [48] without mutual interference. As illustrated in Fig. 3, a frequency-comb source injects a band of wavelengths into a shared waveguide.

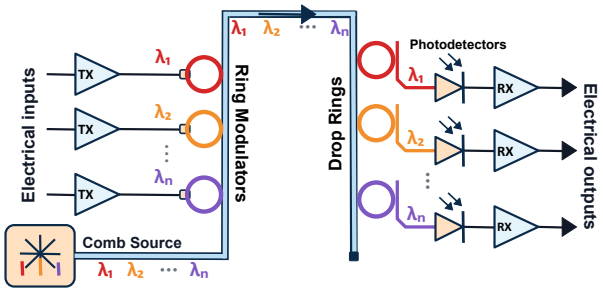


Fig. 3: Illustration of WDM signal modulation, propagation, and receiving.

At the transmitter, the resonance of each ring modulator is aligned with one wavelength from the frequency-comb source. As the multi-wavelength light propagates along the waveguide, each ring independently modulates its assigned wavelength according to the corresponding electrical input. The modulated wavelengths remain multiplexed on the same physical waveguide [30], [77]. WDM therefore aggregates the bandwidth of multiple wavelength channels on each physical waveguide. For example, 32 or 64 wavelength channels operating at 32 Gb/s per wavelength provide aggregate bandwidths of 1 or 2 Tb/s per waveguide, respectively [30], [87].

C. Photonic Integration on Glass Interposers

Panel-level glass interposers provide a mechanically stable platform for large-area die assembly and integrate the electrical and optical infrastructure spanning the package. On such platform, redistribution layers (RDLs) provide lateral routing for power and control signals, through-glass vias (TGVs) provide vertical connectivity through the substrate, and integrated optical waveguides carry high-bandwidth data over long on-substrate distances [7], [10]–[13], [23], [71], [73]. Prior work has demonstrated flip-chip assembly of PICs on glass substrates with co-integrated optical waveguides and electrical redistribution, including cavity-based interfaces that provide both optical coupling and electrical attachment between the xPU and the substrate [7], [10], [27], [56].

An optical engine combines a PIC with the electrical interfaces required for E/O and O/E conversion. Near-packaged optics (NPO) places separately packaged optical engines near the host package, whereas co-packaged optics (CPO) integrates the host and optical engines on the same package substrate [55], [65]. These approaches typically use optical engines to connect host I/O to external fibers. Glass interposers additionally allow optical routing to be integrated into the package substrate itself: locally attached PICs perform E/O and O/E conversion, while embedded waveguides carry optical signals between dies across the package [7], [10], [13]. Recent work integrates reconfigurable switching directly into the photonic layer of a panel-scale glass interposer [30].

III. GlassML Design

A. GlassML Overview

GlassML enables a panel-level glass interposer with an embedded optical fabric to serve as a scale-up domain for LLM training. Fig. 4(a) provides a design overview. GlassML

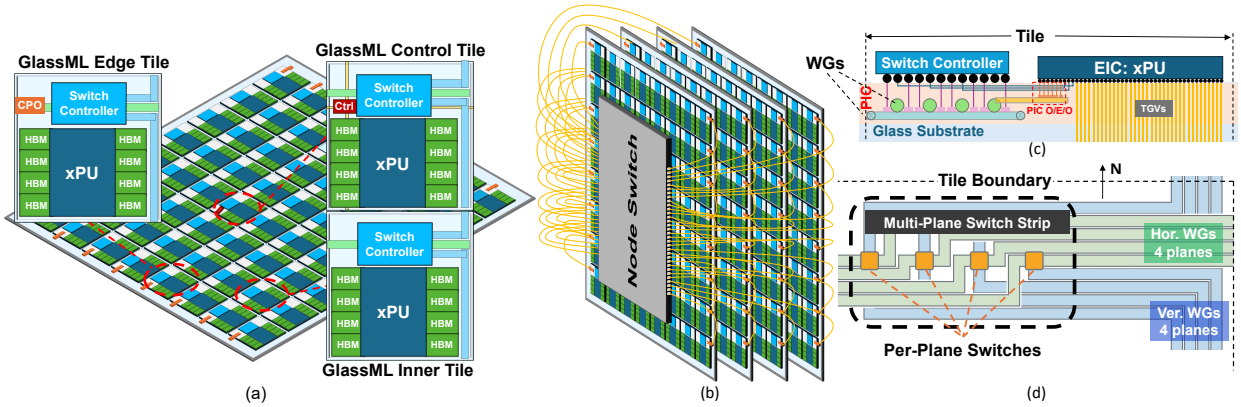


Fig. 4: (a) Overview of GlassML, a panel-level glass interposer with embedded reconfigurable optical fabric. Horizontal and vertical WGs span rows and columns across panel, and intersect at every tile under the switch controller. On the tile, WGs are routed to the side to avoid dense Through Glass Via (TGVs) under the xPU. (b) Multi-panel setup for further scale out via an external switch. (c) Side view of a GlassML tile. (d) Top-down view of the multi-plane switch laid out as a switch strip, where each switch connects only to the WGs in its plane. Horizontal WGs are routed one layer above vertical WGs.

integrates an $n \times n$ array of compute tiles on the interposer, each holding an xPU with its local high-bandwidth memory. Optical waveguides embedded in the glass substrate span the panel and form a reconfigurable circuit-switched fabric among the $N = n^2$ tiles. Each tile carries an optical switch where the horizontal and vertical waveguides of its row and column intersect, and a circuit between any pair of tiles is formed by configuring the switches along its path. Multiple GlassML panels can be interconnected through optical fibers and an external switch for further scale-out, as shown in Fig. 4(b). Table I lists the notations used in this paper.

Architectural design point. As a concrete instance, we consider a 600×600 mm panel with $n = 8$, yielding $N = n^2 = 64$ tiles, each pairing a B300-class xPU with 288 GB of HBM3e [63]. At $K = 16$ planes and 2 Tb/s per WG, the panel thus constitutes a single scale-up domain of 64 GPUs, 448 PFLOPs FP8 and 18.4 TB of HBM, interconnected by an optical fabric at 4TB/s max P2P bandwidth.

B. Tile Architecture

A tile is the basic unit of construction on the panel-level glass interposer, serving at once as a compute rank, as an endpoint that injects into and receives from the optical fabric, and as one switching node of that fabric. Each tile pairs electronic dies with a photonic layer embedded in the glass substrate beneath them. The electronic layer holds an electronic integrated circuit (EIC), a separately manufactured KGD such as an xPU with its local memory, which is flip-chip bonded through micro-bumps onto the micro-pads of the photonic layer [30]. Beneath the electric layer, the photonic layer constitutes a panel-wide photonic integrated circuit (PIC) containing the electrical-optical-electrical (E/O/E) conversion interfaces and the tile optical switch [30], which establishes reconfigurable through and turning paths between the horizontal and vertical waveguides that intersect at the tile. On the transmit path, data descends from the die through the micro-bump and the PIC metal and via stack and is encoded onto an optical carrier in the waveguide; the receive path performs the inverse conversion through a destination tile photodetector.

We introduce three kinds of tiles that collectively support panel control, intra-panel optical communication, off-panel communication, and the compute functionalities required to operate an LLM training scale-up domain.

GlassML Inner Tile. Inner tiles constitute the majority of the panel and implement the common structure alone. Each operates as one rank, as a GPU does in a conventional cluster, and its switch couples the horizontal and vertical trunk WGs of its row and column.

GlassML Edge Tile. Edge tiles occupy the east and west boundaries and additionally integrate co-packaged optics (CPO) that terminate the on-panel WGs and drive optical fiber. Inter-panel traffic is first routed to an edge tile over the on-panel fabric and is then handed to the CPO transceiver.

GlassML Controller Tile. The controller tile sits at the center of the panel and additionally integrates a panel controller ASIC, which collects the communication demand of the ranks, solves the routing and scheduling problem of §IV in hardware, and distributes the resulting switch configurations.

Every tile is allocated one pair of control WGs to and from the controller tile. These are routed on a separate layer of the interposer and therefore do not contend with the data WGs that carry payload traffic, so the control domain is exempt from the routing and scheduling problem that governs the data domain.

C. Optical Switching Element

Ring resonators provide the basic optical switching mechanism used in GlassML. As shown in Fig. 5(b), when an incoming optical carrier aligns with a ring resonance (ON state), it couples onto the drop waveguide; when the ring is detuned (OFF state), the carrier remains on the input waveguide and passes through the device. An integrated microheater controls the switching state by thermo-optically shifting the ring resonances into or out of alignment with the carrier. At each crosspoint, the orthogonal waveguides are routed on vertically separated photonic layers. A vertically coupled add-drop micro-ring resonator (VC-AD-MRR) placed between them can therefore redirect light from one waveguide to the

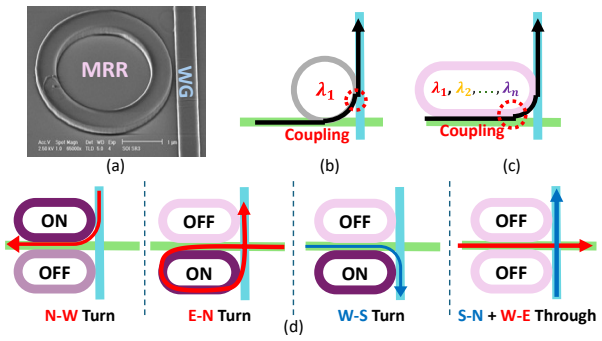


Fig. 5: (a) Microscopic view of a micro-ring resonator (MRR), adapted from [84]. (b) A vertical-coupled add-drop micro-ring resonator (VC-AD-MRR). (c) A vertical-coupled add-drop racetrack resonator (VC-AD-RTR). (d) A crosspoint realized with two racetracks and example turning capabilities.

TABLE I: Notations of Panel-Level Glass Interposer

Symbol	Meaning
n	Number of tiles in a row/column on the panel
N	Total number of tiles on the panel
$Tile_{i,j}$	Tile at row i, column j
$H_{i,j}$	The j-th horizontal trunk WG of row i
$V_{i,j}$	The j-th vertical trunk WG of column i
$A^{\mathrm{TX}}_{i,j}$	TX access WG of $Tile_{i,j}$
$A^{\mathrm{RX}}_{i,j}$	RX access WG of $Tile_{i,j}$
K	Number of parallel planes
δ	Reconfiguration time

other. A conventional micro-ring resonator typically operates on an individual wavelength. To switch an entire WDM signal as one unit, the micro-ring is elongated into a racetrack shape, adjusting its cavity length so that its free spectral range matches the channel spacing of the optical comb. The resulting vertically coupled add-drop racetrack resonator (VC-AD-RTR) aligns its resonances with all WDM channels simultaneously [30]. We refer to it as *racetrack* for short. Two racetracks per crosspoint are required to enable all turning directions, as shown in Fig. 5(d). GlassML thus switches at waveguide granularity including all WDM channels.

D. Optical Switching Fabric

A single crosspoint with two racetracks forms a switch for two intersecting WGs. To build an optical switching fabric for the entire panel, we start by constructing a grid of such crosspoints to form an optical crossbar over the intersecting horizontal and vertical WGs. With n^2 crosspoints, the crossbar establishes up to n concurrent lightpaths between the n horizontal and vertical WGs. Then, we allocate each tile one such crossbar to form a panel-level optical mesh.

We note that, while the optical fabric intuitively forms a mesh network on the glass interposer, it differs fundamentally from an electronic mesh in its allocation granularity and resource sharing paradigms. In an electrical mesh, routers buffer and re-drive every signal, partitioning each row into independent wire segments that distinct flows occupy and time-share separately. The optical fabric performs no buffering: a transfer requires a complete lightpath, configured and held for its entire duration, and the same switching element cannot be shared between two lightpaths due to interference. With these constraints, sustaining arbitrary permutation traffic on such a

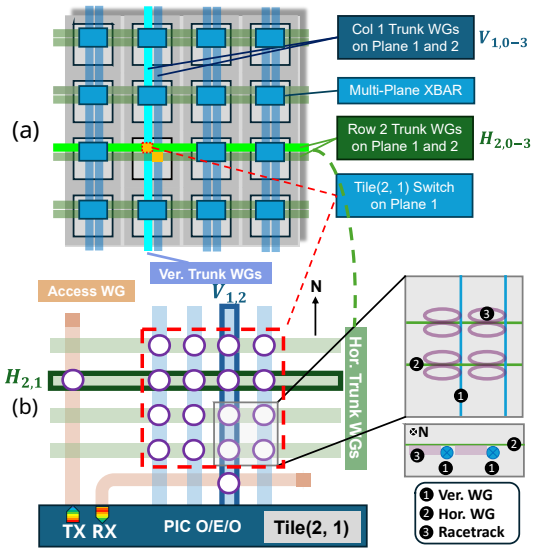


Fig. 6: (a) A fabric-centric view of a panel with 2 switching planes, shown as two bundles of trunk WGs per row and column. (b) A focused view on the crossbar switch on a $Tile_{2,1}$ at plane 1. $Tile_{2,1}$ *owns* the accented vertical and horizontal trunk WGs, $H_{2,1}$ and $V_{1,2}$. Not to scale.

fabric requires n trunk WGs per row and per column, **one associated with each tile**, which we provision and later prove to be both necessary and sufficient with Theorem 1.

Fig. 6 illustrates a 4×4 panel with two planes, and focuses on the switch architecture of $Tile_{2,1}$ of a single plane. We consider two kinds of waveguides. *Trunk WGs* span entire rows and columns to carry traffic across the panel: the n horizontal trunk WGs of each row and the n vertical trunk WGs of each column are embedded in two separate glass interposer layers and intersect at the $n \times n$ crossbar inside every tile switch for their plane. *Access WGs* are local, short-reach WGs that connect the tile to the fabric: the TX access WG carries modulated carriers from the tile E/O interface into the fabric, whereas the RX access WG delivers arriving carriers to the tile photodetectors. The access WGs are the tile's only physical entry and exit in the plane, each providing the bandwidth of a single WG to the xPU's send and receive paths. We define a **switching plane** as n trunk WGs per row and column, one $n \times n$ crossbar switch per tile, one pair of access WGs per tile, none of which are shared between planes.

Therefore, with a single plane, the per-xPU bandwidth is limited. To meet the bandwidth requirement of a modern accelerator, each xPU must drive K TX/RX pairs, such that $\mathrm{BW}_{\mathrm{target}} = K \cdot \mathrm{BW}_{\mathrm{WG}}$. These WGs and their switches can be organized in two ways. The first is a monolithic crossbar that incorporates all WGs at each tile: each row then carries nK horizontal WGs and each column nK vertical WGs, so every tile switch becomes an $nK \times nK$ crossbar containing $2(nK)^2$ racetracks, a count that grows quadratically with K. Considering 1 Tbps per WG [30], $n = 8$ and $K = 8$ provides 2 TBps reversible (half-duplex) bandwidth for 64 GPUs, similar to an NVL72 system [63], requiring 8192 racetracks per tile switch.

Taking a different approach, GlassML replicates the single-

plane fabric into K independent parallel switching planes. Each per-tile switch then comprises K independent $n \times n$ small crossbars for all planes, containing $2Kn^2$ racetracks in total per tile, a factor-K reduction in racetrack count. In addition, the amount of optical loss from passing through racetracks is significantly reduced. The same $n = 8$, $K = 8$ configuration now requires 1,024 racetracks per tile to provide the same bandwidth. In addition, Fig. 4(d) depicts the resulting multi-plane switch. Different switching planes share no WGs or crosspoints and are configured independently, whereby the K planes equivalently multiply the fabric bandwidth by a factor of K. In addition, as shown in Fig. 4(d), per-plane switches can be arranged without planar conflict by turning the WGs at loss-free radius. The resulting multi-plane optical crossbar is constructed as a switch strip of per-plane switches in the tile, as seen in Fig. 4(d), which further reduces the area footprint by a factor of K compared to a monolithic implementation.

IV. GlassML Routing and Scheduling

LLM execution, whether training or inference, comprises compute and collective communication operators. Realizing a collective operator on the optical fabric introduced in §III fundamentally differs from realizing it on an electrical mesh. While an electric router resolves the next hop of a packet locally and on arrival, a circuit must be established end to end before transmission begins and be held for its entire duration, and circuits that contend for the same resources cannot coexist. This mandates solving for routing before communication time.

We approach this question by decomposing the problem into three consecutive sub-problems: (1) formulating collective operators as demand matrices (§IV-A), (2) resolving that matrix into a sequence of fabric reconfigurations (§IV-B), and (3) realizing them as a set of conflict-free circuits across the panel that do not need more WGs than provided (§IV-C).

A. Formulating Collectives as Demand Matrices

Each collective communication operator can be expressed by a demand matrix $D \in \mathbb{R}_+^{N \times N}$, where $D_{i,j}$ is the payload in bytes that tile i must deliver to tile j (out of N tiles). To do so, there are two main possible approaches: either formulating D in a predetermined manner, e.g., for TP/DP/PP parallelisms, or doing so at runtime, e.g., for EP (MoE).

Predetermined collectives. The collectives induced by tensor and context parallelism communicate along fixed groups of ranks, so their demand follows from the parallelism strategy and is available before execution begins. Specifically, in ring AllGather and ReduceScatter over a group of ranks, rank i transmits to rank $i + 1$ at every step. In addition, a ring AllReduce is a ReduceScatter followed by an AllGather and preserves the same mapping. Therefore, all demand matrices other than EP AllToAll can be precomputed, together with a full schedule of permutations in each plane.

Runtime-determined collectives. The router of each MoE layer assigns tokens to experts from the layer input, so the demand matrix is only determined at run time. It is also dense: with the experts distributed over all N ranks, a rank holds tokens for nearly every other, and the degree of D approaches N. These operators therefore require the full solution of §IV-B, and they account for nearly all of the reconfigurations a schedule incurs. Dispatch and combine of the same MoE layer carry equal payload in opposite directions, therefore the combine phase can reuse the dispatch schedule transposed, and one solution serves both phases.

B. Scheduling

Decomposition into permutations. GlassML provides K switching planes that share no WGs or crosspoints and reconfigure independently. Each plane $k \in \{1, \ldots, K\}$ schedules its own sequence of m^k permutation matrices P_l^k, i.e., a sequence of consecutive concurrent circuits, where $P_l^k(i, j) = 1$ when a circuit from tile i to tile j is established. A tile is allocated one TX and one RX access WG per plane (§III-D), so,

$$\sum_j P_l^k(i,j) \le 1 \quad \forall i, \qquad \sum_i P_l^k(i,j) \le 1 \quad \forall j, \tag{1}$$

Each of these m^k permutation matrices P_l^k is assigned a duration α_l^k, and each of its connections uses the full WG capacity of C. Thus, holding P_l^k for a duration α_l^k delivers $C \cdot \alpha_l^k$ bytes for each connection. Therefore, completing a collective requires the payload that these sequences deliver to satisfy the demand, i.e.,

$$D \le C \sum_{k=1}^{K} \sum_{l=1}^{m^k} \alpha_l^k P_l^k, \tag{2}$$

where the inequality holds entrywise (i.e., the demand between each pair of sender and receiver tiles is satisfied).

CCT. We assume that at each plane, setting P_l^k always requires a reconfiguration delay δ, during which the plane carries no traffic. Therefore, the completion time of plane k is $\sum_l (\delta + \alpha_l^k)$, and the Collective Completion Time (CCT) is that of the slowest plane,

$$\mathrm{CCT} \;=\; \max_{1 \le k \le K} \; \sum_{l=1}^{m^k} \left(\delta + \alpha_l^k\right). \tag{3}$$

Scheduling algorithm. Minimizing Eq. (3) subject to Eq. (2) is NP-hard even for a single plane [41]. Therefore, we search for a practical approximation algorithm. We extend the polynomial-time SPECTRA algorithm [42], which was designed to find a schedule over several optical switches, and adapt it to our problem of scheduling over several optical planes. *Our key observation for the correctness of* SPECTRA *is that the mathematical settings are the same:* each of the planes is independent, and each assigns consecutive permutations with a reconfiguration cost. Thus, we can independently assign permutations to each plane without being constrained by other planes, in the same way as SPECTRA assumes that each optical switch schedules permutations independently.

Example. Fig. 7 illustrates a full example of demand matrix decomposition and routing. It focuses on a 2×2 demand matrix D between $N = 4$ tiles, and shows a possible decomposition

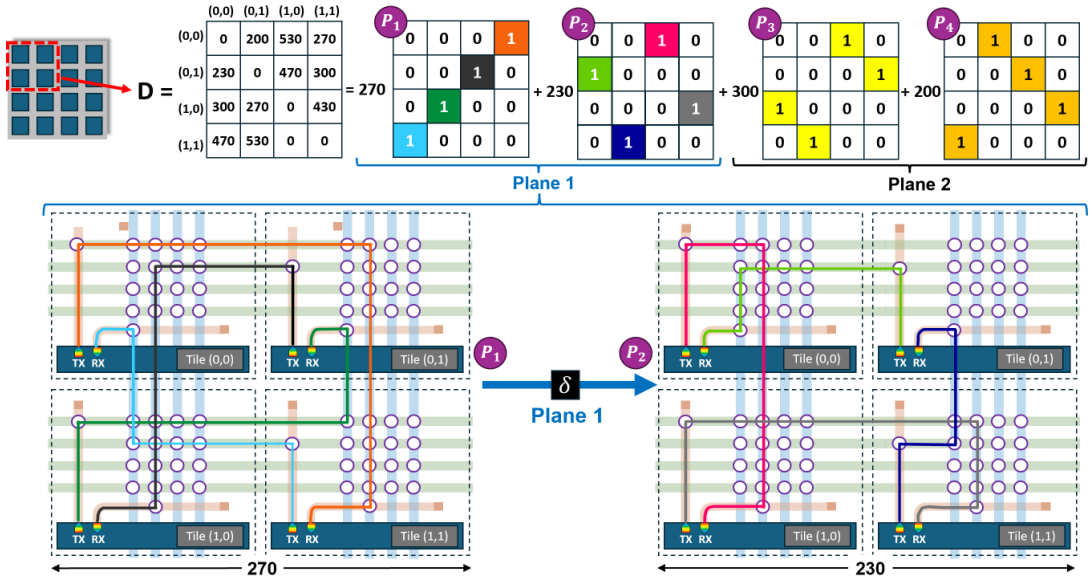


Fig. 7: *Full example of demand matrix decomposition and routing.* The 4×4 demand matrix D corresponds to the upper left 4 tiles of a 4×4 panel (dashed red square), assuming no other demands. D is then decomposed into 4 weighted permutations, which are assigned to 2 planes. Zooming in on the 2×2 implementation in plane 1 shows how each of the first two permutations are physically routed using XY routing, with a reconfiguration time δ between them.

into the sum of four weighted permutations. For example, in the first permutation, the first tile sends traffic to the fourth (orange element), the second to the third (black), etc. The permutation weight is 270. Note that here the decomposition is exact for simplicity.

Once we obtain this decomposition, we assign each permutation to a plane. To obtain the exact durations that correspond to the weights in the figure, We divide the weight of each permutation (in MB) by C (in MB/s). For example, if the weight of the first yellow permutation is $270\,\text{MB}$ and $C = 0.5\,\text{Tbps}$, then we obtain a duration of $270/0.5 = 540\,\mu\text{s}$. As a result, the schedule in the first plane will be (1) reconfiguration for a duration of δ, (2) first permutation for $540\,\mu\text{s}$, (3) reconfiguration (δ), and finally (4) second permutation for $460\,\mu\text{s}$. If $\delta = 50\,\mu\text{s}$, we obtain $CCT = 1.1\,\text{ms}$.

C. Routing

Routing challenges. The scheduling step above provides us with a sequence of permutations in each plane. However, once we know which tile i should send traffic to which tile j, we are left with the question of how to route this traffic on the reconfigurable optical fabric.

We need to solve two challenges when designing our routing algorithm: (1) our routing needs to be simple enough to be easily implemented in practice; for example, we would like to use a simple oblivious and deterministic routing like XY routing (also known as Dimension-Order Routing [19]), which routes packets first along the X-axis until the horizontal coordinate matches the destination, and then along the Y-axis until the vertical coordinate also matches. In addition, (2) our routing algorithm also needs to be efficient in WG count. For example, if our routing is inefficient, we may have many flows going through a single link, and therefore we may need to provision for many WGs, strongly increasing the cost of our architecture.

Optimality of XY routing. We now consider an $n \times n$ mesh network where n is even. We establish two key properties: (i) regardless of what routing algorithm we use, we need to provision at least n WGs per link, and (ii) the simple XY routing is optimally guaranteed to fit within these n WGs.

Theorem 1 (XY Routing Optimality). *In an $n \times n$ mesh network with even n, reversible links, n_0 WGs per link:*
(i) To route any arbitrary permutation traffic pattern, an optimal routing needs to provision at least n WGs per link, i.e., $n_0 \geq n$.
(ii) XY routing is optimal, i.e., it is guaranteed to route any arbitrary permutation when there are $n_0 \geq n$ WGs per link.

Proof Sketch. (i) Consider a vertical bisection cut dividing the mesh symmetrically into two equal parts. In a worst-case permutation where the tiles of each part send to the other part, n^2 flows need to cross the cut, i.e., we need at least n^2 WGs across the n reversible links in the cut. Thus, each link needs at least $n^2/n = n$ WGs.

(ii) Under fixed XY routing for an arbitrary permutation, a horizontal reversible link L in row r carries only flows originating from nodes in row r. Since there are at most n nodes in row r, there can be at most n flows, hence the load of L is at most n. Same for a vertical link L', which can only carry flows terminating in the same column as L', of which there are at most n. Hence the result. □

Realizing permutations. Theorem 1 establishes that n trunk WGs per row and per column are sufficient and necessary to sustain arbitrary permutations. Here, we introduce the mapping between permutation and fabric resources through an ownership model between trunk WGs and tiles. Within each

plane, $Tile_{i,j}$ owns the horizontal trunk WG $H_{i,j}$ of row i and the vertical trunk WG $V_{j,i}$ of column j. Importantly, the owned horizontal trunk WG constitutes the dedicated egress and the owned vertical trunk WG constitutes the dedicated ingress of the tile, whereby every lightpath originating at $Tile_{i,j}$ enters the fabric on $H_{i,j}$ from $A^{\mathrm{TX}}_{i,j}$ and every lightpath terminating at $Tile_{i,j}$ leaves the fabric on $V_{j,i}$ for access WG $A^{\mathrm{RX}}_{i,j}$. For example, Fig. 6 shows that $Tile_{2,1}$ owns $H_{2,1}$ and $V_{1,2}$, serving as its sole egress and ingress within that plane.

Theorem 1 also establishes that XY routing is optimal, and the ownership model realizes it without further constraint. From the perspective of a plane, a lightpath from source $Tile_{i,j}$ to destination $Tile_{k,l}$ must depart on $H_{i,j}$, the dedicated source egress, and must arrive on $V_{l,k}$, the dedicated destination ingress. Since $H_{i,j}$ spans row i and $V_{l,k}$ spans column l, the two trunk WGs cross at a single crosspoint, located in the switch box of $Tile_{i,l}$, which is therefore the only turning point of the lightpath on the mesh fabric. This establishes a one-to-one mapping between any permutation to unique switch configurations on the panel-level interposer.

Example. Fig. 7 illustrates the routing in the 2×2 upper-left section of a 4×4 panel, given the two consecutive permutations in plane 1. In the first configuration provided by the first permutation, the orange lightpath carries the flow from $Tile_{0,0}$ to $Tile_{1,1}$: the carriers are injected from $A^{\mathrm{TX}}_{0,0}$ onto $H_{0,1}$ in the eastbound direction, turn southbound onto $V_{1,1}$ in the switch box of $Tile_{0,1}$, and terminate on $A^{\mathrm{RX}}_{1,1}$.

D. Execution

1) Intra-panel Communication: In MoE execution, once every rank has completed its router module and holds tokens destined for experts on other ranks, each rank reports its per-destination byte counts to the controller tile using WGs in the control domain, which involves no scheduling or routing. The controller then assembles the demand matrix, solves it with the steps of §IV-B, and dispatches the resulting permutations and durations to all ranks. Upon receiving the schedule, the switch controller of each tile configures its crosspoints for the first assigned permutation and holds each subsequent configuration for its assigned duration, while the rank transmits toward the destination prescribed by the active permutation. Between consecutive permutations, reconfiguration delay δ is spent to tune racetracks for the next states.

2) Inter-panel Communication: In LLM training, the demand of cross panel communication, such as pipeline parallel (PP) inter-stage activation transfer, can be derived from the job mapping and is therefore known before execution. Cross-panel transfers reach their edge tile over a fixed configuration as they reside inthe same row, therefore no scheduling or reconfiguration is required. The edge tiles buffer and then retransmit through their CPO transceivers into inter-panel fibers, whose topology is likewise configured once at job launch to match the PP stage adjacency. This suffices because cross-panel volume is far below that of the intra-panel collectives, since EP and CP are confined within a panel while PP exchanges only stage boundary activations.

TABLE II: System Configurations for a 64-GPU Scale-Up Domain

System	Organization	Per-GPU	Network
Waferscale [79]	4×(4×4) mesh	8 TB/s	Mesh, 9 TB/s per border
NVL64 [63]	Subset of NVL72	1.8 TB/s	Non-blocking
GlassML-K8/1T	Panel-level GIP	2 TB/s	8 planes, 1 Tb/s/WG
GlassML-K8/2T	Panel-level GIP	4 TB/s	8 planes, 2 Tb/s/WG
GlassML-K16/2T	Panel-level GIP	8 TB/s	16 planes, 2 Tb/s/WG

V. Evaluations

A. Hardware Configurations

We consider a panel-level GIP of size 600×600 mm [18], divided into 64 tiles as a 8×8 grid. We consider each tile xPU as an NVIDIA B300 GPU with 288GB of HBM3e memory [63], with one modification to its I/O. The NVLink beachfront at the top of the die is repurposed to drive the modulators in the photonic layer immediately beneath it, over a sub-millimeter connection that retains the shoreline density of die-to-die interfaces rather than that of the long-reach SerDes (NVLink PHY) it replaces [30], [45]. We assume a 10μs reconfiguration latency of racetrack resonators [24]. To show the advantage of GlassML, we evaluate different setups of 64-GPU scale-up domain, as detailed in Table II. We provide two SoTA baselines: A waferscale chip using a mesh fabric [79]; and a rack-scale NVL64 system (subset of a NVL72 rack) [63]. In addition, we compare three GlassML design points with different numbers K of parallel planes and different speeds per WG.

B. Experimental Setup

Simulator setup. We build our simulation platform based on Microsoft Apex Plus [46] to model Megatron-style LLM training [58]. For a given model, workload, and parallelism, the simulator first constructs a graph of compute nodes and communication nodes by partitioning the work across the distributed GPUs. The total workload latency is the sum of the cost of all nodes in the graph. For compute operators, we adopt a profile-then-simulate approach like prior work [79] by running SoTA implementations of LLM operators such as Flash Attention 4 [90] and CuBLAS [61] on B300 GPUs. The profiling results are used to construct the look up table.

For communication operators on GPU baseline, we use measurement from GPU clusters to create a look up table. We use the NCCL test suite [62] to profile all collective communication (AllReduce, AllToAll, ReduceScatter, AllGather). For intra-node communications, we profile NVL72 (using 64 GPUs within), and for cross-node communication, we measure the performance of clusters with one ConnectX-7 400Gbps NIC per GPU for InfiniBand connection, adhering to typical datacenter setup.

For communication operators non-GPU baselines of waferscale and GlassML, we implement our own fabric simulator. The GlassML fabric simulator contains models for optical WGs, overheads for control domain communication, circuit reconfiguration, and costs of SPECTRA solver. We assume all systems have the same software overhead when launching communication kernels, as profiled from NCCL.

TABLE III: Model and workload configurations.

Model	Params (Total/Act.)	Experts (Routed/Total)	Hidden	Layers
Qwen3-30B [25]	30B / 3B	8 / 128	2048	48
Qwen3-235B [25]	235B / 22B	8 / 128	4096	94
Ernie4.5-300B [5]	300B / 47B	8 / 64	8192	54

Training Configurations (mbs=1)

Model	Seqlen	Attn. CP	MoE EP	PP	#GPUs	Recompute
Qwen3-30B	8K	1	64	1	64	-
	32K	2	64	1	64	-
	128K	8	64	1	64	-
Qwen3-235B	8K	2	64	2	128	-
	32K	1	64	2	128	Full
	128K	2	64	2	128	Full
Ernie4.5-300B	8K	2	64	2	128	-
	32K	1	64	2	128	Full
	128K	2	64	2	128	Full

MoE workloads. We evaluate training workloads of the following three MoE models: Qwen3-30B-A3B, Qwen3-235B-A22B [25], and Ernie-4.5-300B-A47B [5]. We consider three training stages with sequence lengths 8K, 32K, and 128K tokens per microbatch, respectively, reflecting the common practice of progressively increasing the context length during training to improve the long-context capabilities of the models [21], [25], [36]. Table III details our workload settings, including training parallelism and recompute needed to fit the model weights, optimizer states, and activations. We only apply pipeline parallelism (PP) across scale-up domains (64 GPUs). We use the folded parallelism introduced in [47] to decouple parallelism for experts and attention. For example, on 64 GPUs, experts can be sharded 64 ways while KVs are sharded 2 ways by sequence length, i.e. MoE EP64 + Attn CP2. We calculate $T_{\text{iter}} = T_{\text{fwd}} + T_{\text{bwd}}$, and use it to approximate training throughput, assuming PP-stage latency can be well hidden [86].

MoE considerations. EP AllToAll traffic skew is a first-order characteristic of EP training because token-to-expert assignments are runtime-dependent, and is critical to communication fabric performance. We model three cases: (1) *uniform distribution*, (2) *replayed skewness*, and (3) *high-skew Zipfian*, to evaluate the effect of load imbalance among EP ranks across systems. For replayed skewness, because intermediate training checkpoints are unavailable, we instrument teacher-forced forward passes through the released checkpoints of three models to record the token destinations selected by each MoE router. The collected per-layer routing distributions are then injected into the simulator to reflect the EP load balancing. We process 1M tokens from FineWeb [67], UltraChat [22], and PG-19 [68] at 8K, 32K, and 128K sequence lengths, respectively, and observe average maximum-to-average token-dispatch ratios of 3.64×, 4.21×, and 6.34× across 64 EP ranks for Qwen3-235B-A22B. For high-skew Zipfian, we use a Zipfian distribution with $s = 0.65$ to synthesize MoE AllToAll token dispatch with an 8× maximum-to-average ratio, following prior work [42], [43]. We map experts to EP ranks by assigning contiguous expert IDs to each GPU rank.

C. Microbenchmarks

We evaluate four 64-rank collective communication microbenchmarks: Ring AllGather; and AllToAll under uniform, measured, and Zipfian expert-routing distributions. Fig. 8 reports the CCTs of Waferscale [79], NVL64 [63] and three GlassML configurations, normalized to that of Waferscale. Due to higher bandwidth and reconfigurability, GlassML's AllToAll advantage increases with message size because the fixed solve and reconfiguration costs are increasingly amortized: under uniform traffic, GlassML-K16/2T improves from 4.20× at 8K tokens per rank to 8.19× at 128K. Skew further amplifies this advantage because hot communication pairs congest Waferscale's fixed mesh, whereas GlassML reconfigures its circuits to retain max bandwidth. Over NVL64, under uniform traffic, GlassML-K16/2T improves from 2.89× at 8K tokens per rank to 5.57× at 128K. At 128K, the speedup further reaches 5.84× under measured skew and 5.92× under the Zipfian. Ring AllGather allows Waferscale to more effectively utilize its bandwidth, where it nearly matches the NVL64 baseline. The Solve+Reconfig tax is up to 46% of the K16/2T CCT at shorter, 8K tokens under uniform routing, but quickly diminishes to 5.7% at 128K, and to 1% under measured skew at 128K. Higher skew inflates pure bandwidth time, which reduces the Solve+Reconfig tax rate.

D. LLM Training Throughput

Fig. 9 illustrates the normalized training speedup under the same workload mappings. Under uniform routing, NVL64 reaches up to 1.06× over Waferscale, GlassML-K8/1T up to 1.10×, and GlassML-K16/2T up to 1.24×. With balanced traffic, communication constitutes a smaller fraction of the iteration, limiting the benefit of additional bandwidth due to Amdahl's Law. Under measured skewness, GlassML-K8/1T reaches up to 1.31× with a 1.26× geometric mean across the two Qwen models, but only up to 1.13× on Ernie-4.5, whose more uniform MoE routing induces less communication imbalance. Across all nine measured workloads, GlassML-K16/2T achieves up to 1.40× and 1.26× speedup over Waferscale and NVL64, respectively, with corresponding geometric-mean speedups of 1.27× and 1.18×. Under the synthetic Zipfian distribution, GlassML-K16/2T similarly reaches up to 1.40× with a 1.27× geometric mean, indicating that the system benefit has largely saturated at the skewness observed in the measured workloads. At higher skewness, expert computation on the most-loaded rank becomes the bottleneck, which additional interconnect bandwidth cannot reduce. Finally, the smaller speedups at longer sequence lengths are consistent with attention accounting for a larger fraction of the iteration.

E. Ablation studies

Reconfiguration cost. In Fig. 10, we increase the racetrack-resonator reconfiguration delay δ from $10\,\mu$s to $50\,\mu$s to study its effect on AllToAll and training throughput. For the measured Qwen3-235B routing distribution, increasing δ reduces the GlassML-K8/1T AllToAll speedup over Waferscale from 1.96× to 1.71× at 8K tokens, from 2.14× to 2.07× at 32K,

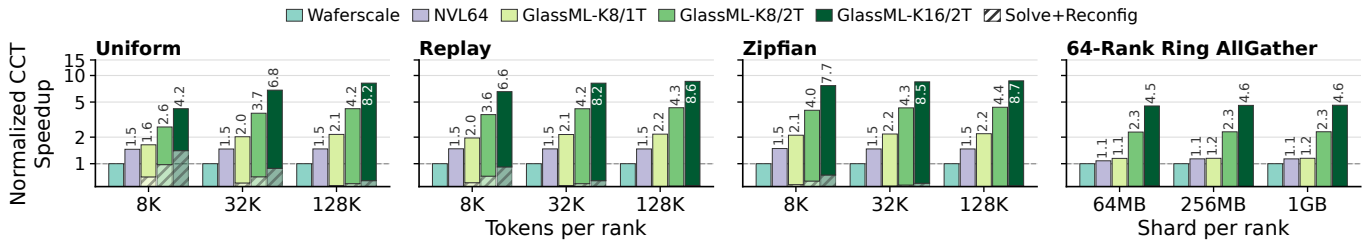


Fig. 8: *CCT speedup* normalized to Waferscale [79], using collective communication microbenchmarks of Qwen3-235B EP64 AllToAll and Ring AllGather. The striped portion of each GlassML bar shows the fraction of the CCT spent on schedule solving and reconfigurations, relative to full height of the bar.

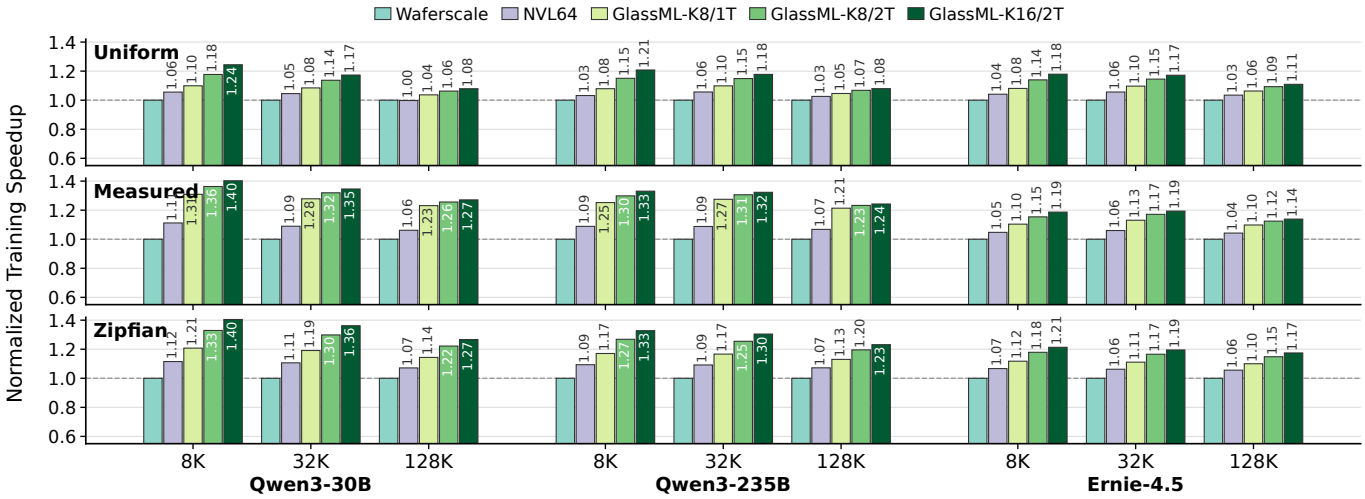


Fig. 9: *Training speedup* across the three models and five systems of Fig. 8.

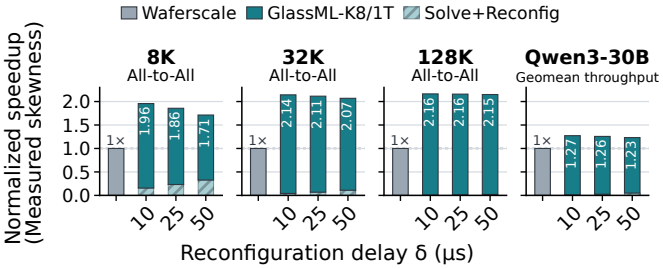


Fig. 10: Ablation of reconfiguration cost on GlassML-K8/1T. The striped portion of each GlassML bar shows the fraction of the CCT spent on schedule solving and reconfigurations, relative to full height of the bar.

and from 2.16× to 2.15× at 128K. Relative to NVL64, the corresponding speedups decrease from 1.33× to 1.16×, from 1.46× to 1.41×, and from 1.47× to 1.46×, respectively. At the worst-case 50 μs delay, the Solve+Reconfig tax accounts for up to 19% of communication time at 8K but only 1% at 128K because of the much larger dispatch size. Consequently, across the three sequence lengths, Qwen3-30B retains geometric-mean training speedups of 1.23× over Waferscale and 1.13× over NVL64 at 50 μs, compared with 1.27× and 1.17×, respectively, at 10 μs. These results show that larger collectives in long-context training preserve nearly all of GlassML's bandwidth benefit.

F. Overhead and Cost Analysis

Area. We assume that each tile measures 71.5×71.5 mm. A B300 GPU package consists of two stitched 26×33 mm reticles and two HBM3e stacks of size 11×11 mm per 26 mm side of each reticle [52]. Waveguide pitch is conservatively 50 μm [30]. In routing, we consider four layers of WGs for vertical trunk WGs, horizontal trunk WGs, access WGs, and finally control WGs, with 8 layers supported in GIP [11]. Each racetrack resonator takes 0.04 mm^2 using serpentine routing [59]. We consider 100 μm turning radii for WGs for negligible loss [40]. We assume the glass interposer area under the xPU die is used for through-glass vias (TGVs) for power supply and low-speed signals and therefore cannot be used for waveguide routing. Hence, in actual floorplan, the xPU resides in the lower left corner of the tile, and horizontal and vertical trunk WGs are routed above and to the right of the xPU die. The switch strip is located above the xPU, and the vertical trunk WGs make a C-shape turn to access the switch strip, before turning north to leave the tile. We present an example layout in Fig. 11. We note that the WGs can be routed at much lower pitch to improve total bandwidth, and more complex and multi-layer layout can be explored.

Scheduling Overhead. We design an AVX512 CPU kernel for SPECTRA on AMD ThreadRipper 9985WX and measure its runtime as 300μs using a single core. We estimate that with a dedicated control domain and an ASIC solver, the total

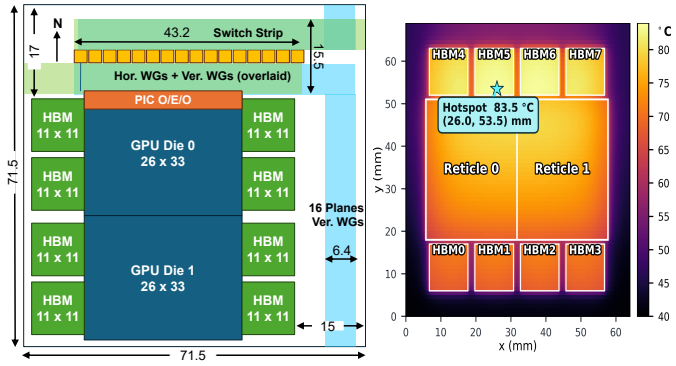


Fig. 11: Left: example layout of 16-plane GlassML (unit: mm). Right: 3D-ICE [81] simulation of B300 GPU at full TDP on glass interposer.

scheduling overhead can be 100μs. In our simulation, this cost is applied only to MoE dispatch stage, as the combine stage's demand matrix is the transpose of that of the dispatch stage.

Optical Loss. Under XY routing, the worst-case intra-panel lightpath travels from corner to corner and traverses 16 per-plane 8×8 crossbars, resulting in 144 racetrack resonator crossings. The ownership model of §IV fixes exactly three of these in the ON state whereas the remaining 141 are OFF. Taking 0.01 dB per pass-by (OFF), 0.5 dB per resonant coupling (ON), and 0.0177 dB/cm of SiN waveguide [30], [76] loss over a conservatively rounded 200 cm path, the worst-case loss is 141×0.01 + 3×0.5 + 200×0.0177 = 6.5 dB. Closing this budget against a -10.5 dBm receiver [30] requires 0.39 mW per WDM carrier, or ∼65 W of panel-wide carrier generation at 10% comb efficiency, about *0.07% of panel compute power*. Note that *the multi-plane organization keeps this loss independent of system bandwidth*, whereas a single-plane monolithic switch design dramatically increases the number of pass-through racetracks and thus the en-route loss.

Thermal Analysis. The glass interposer's low thermal conductivity of approximately $1\,\mathrm{W/mK}$ limits downward heat flow and in fact favors heat removal through the cold plate on top [82]. We consider standard datacenter liquid cooling solution, under full GPU TDP of 1400 W, using a coolant inlet temperature of $40\,^{\circ}\mathrm{C}$, our 3D-ICE [81] simulation shows peak HBM and logic temperatures of $83.5\,^{\circ}\mathrm{C}$ and $79.7\,^{\circ}\mathrm{C}$, respectively, below the corresponding limits of memory and logic at $85\,^{\circ}\mathrm{C}$ and $105\,^{\circ}\mathrm{C}$ [28], as shown in Fig. 11. In addition, switch power is negligible by construction considering each plane holds one ON racetrack per circuit, at most 64 across the panel and at most 8 within any single 8×8 switch. At 21 mW per FSR-scale retune [24], a fully loaded plane therefore dissipates ∼1.3 W.

VI. Related Work

Wafer-scale Processors. Prior work co-explores wafer-scale architectures, interconnects, and workload mappings for DNN and LLM training. FRED develops a flexible on-wafer fabric for collective communication [70], while Theseus jointly explores hardware configurations and operator mappings [91]. Other systems exploit the abundant on-wafer compute and bandwidth through specialized scheduling and parallelization for LLM serving and EP MoE inference [29], [79], [88].

Co-Packaged Optics. Existing systems use embedded silicon photonics to provide adaptable, high-bandwidth connectivity and resource disaggregation [26], [50], [51]. These systems employ reconfigurable photonic switches or co-packaged DWDM links to steer bandwidth and compose rack-scale resources. SiP-ML jointly optimizes photonic connectivity, model parallelism, and device placement [35], while MixNet combines electrical packet switching with optical circuit switching for distributed MoE training [43].

Demand Matrix Decomposition. Matrix decomposition of a collective demand matrix is frequently employed to coordinate contention-free, globally synchronized schedules across network fabrics, especially for MoE demands. FAST [38] achieves theoretical schedule optimality by applying classical Birkhoff-von Neumann (BvN) decomposition [6], [85], factoring the traffic demand matrix D into a weighted sum of permutation matrices. In contrast, Chronos [72] prioritizes execution speed by substituting exact BvN factorizations with a fast maximal matching heuristic, accepting a trade-off between absolute scheduling efficiency and lower computational complexity. The above works assume a single sequence of permutations. Conversely, SPECTRA [42] decomposes a demand matrix into several independent sequences of permutations, each sequence being scheduled on an optical circuit switch (OCS) out of several parallel OCSes.

VII. Conclusion

We present GlassML, the first architecture and system design that turns a panel-level glass interposer with an embedded reconfigurable optical fabric into a workload-ready scale-up domain for LLM training. GlassML scales bandwidth through a multi-plane optical crossbar that reduces per-tile switch complexity by a factor of K, enforces conflict-free communication through a waveguide ownership model under which XY routing is provably optimal, and schedules both precomputed collectives and runtime-determined expert-parallel AllToAll traffic over demand matrices. Under measured MoE routing skew, GlassML accelerates AllToAll by up to 8.58× over Waferscale and 5.84× over NVL64. Across nine training workloads, GlassML improves training throughput by up to 40.2% and 26.1% over Waferscale and NVL64, respectively.

REFERENCES

[1] ACM Research, Inc., "Panel-level packaging (PLP) process challenges and solutions," https://www.acmr.com/ai-spurring-growth-of-panel-level-packaging/, Nov. 2024, published November 26, 2024; last modified November 3, 2025. Accessed May 2026.

[2] M. AI, "The llama 4 herd: The beginning of a new era of natively multimodal ai innovation," Apr 2025. [Online]. Available: https://ai.meta.com/blog/llama-4-multimodal-intelligence/

[3] Anthropic, "Claude," https://claude.ai/, 2026, large language model; accessed 2026-05-18.

[4] M. Artetxe, S. Bhosale, N. Goyal, T. Mihaylov, M. Ott, S. Shleifer, X. V. Lin, J. Du, S. Iyer, R. Pasunuru *et al.*, "Efficient large scale language modeling with mixtures of experts," *arXiv preprint arXiv:2112.10684*, 2021.

[5] Baidu ERNIE Team, "Announcing the open source release of the ernie 4.5 model family," https://ernie.baidu.com/blog/posts/ernie4.5/, Jun. 2025, eRNIE Blog, accessed July 29, 2026.

[6] G. Birkhoff, "Three observations on linear algebra," *Universidad Nacional de Tucumán. Revista. Serie A: Matemáticas y Física Teórica*, vol. 5, pp. 147–151, 1946.

[7] V. Boucaud, A. Durand, F. Gianesello *et al.*, "Glass interposer for heterogeneous integration of flip-chipped photonic and electronic integrated circuits," *Applied Physics Letters*, vol. 125, no. 5, p. 054101, 2024.

[8] T. Braun, K.-F. Becker, O. Hoelck, R. Kahle, M. Wöhrmann, L. Boettcher, M. Topper, L. Stobbe, H. Zedel, R. Aschenbrenner, S. Voges, M. Schneider-Ramelow, and K.-D. Lang, "Panel level packaging - a view along the process chain," *2018 IEEE 68th Electronic Components and Technology Conference (ECTC)*, pp. 70–78, 2018. [Online]. Available: https://api.semanticscholar.org/CorpusID:51979405

[9] T. Braun, O. Hölck, M. Obst, S. Voges, R. Kahle, L. Böttcher, M. Billaud, L. Stobbe, K.-F. Becker, R. Aschenbrenner, M. Voitel, F.-L. Schein, L. Gerholt, and M. Schneider-Ramelow, "Panel level packaging – where are the technology limits?" in *2022 IEEE 72nd Electronic Components and Technology Conference (ECTC)*, 2022, pp. 807–818.

[10] L. Brusberg, J. R. Grenier, A. R. Zakharian, L. W. Yeary, S.-H. Seok, J.-H. Noh, Y.-G. Kim, J. Matthies, C. C. Terwilliger, B. J. Paddock, R. A. Bellman, D. W. Levesque, R. M. Force, C. G. Sutton, J. S. Clark, and B. J. Johnson, "Glass platform for co-packaged optics," *IEEE Journal of Selected Topics in Quantum Electronics*, vol. 29, no. 3: Photon. Elec. Co-Inte. and Adv. Trans. Print., pp. 1–10, 2023.

[11] L. Brusberg, S. Whalley, C. Herbst, and H. Schröder, "Display glass for low-loss and high-density optical interconnects in electro-optical circuit boards with eight optical layers," *Optics Express*, vol. 23, no. 25, pp. 32 528–32 540, 2015.

[12] L. Brusberg, S. Whalley, R. Pitwon, F. Faridi, and H. Schröder, "Large optical backplane with embedded graded-index glass waveguides and fiber-flex termination," *Journal of Lightwave Technology*, vol. 34, no. 10, pp. 2540–2551, 2016.

[13] L. Brusberg, M. Yeary *et al.*, "Glass substrate with integrated waveguides for surface mount photonic packaging," *Journal of Lightwave Technology*, vol. 39, no. 4, pp. 912–919, 2021.

[14] C.-H. Chen, M. A. Seyedi, M. Fiorentino, D. Livshits, A. Gubenko, S. Mikhrin, V. Mikhrin, and R. G. Beausoleil, "A comb laser-driven DWDM silicon photonic transmitter based on microring modulators," *Optics Express*, vol. 23, no. 16, pp. 21 541–21 548, 2015.

[15] C.-H. Chien, C.-K. Lee, H. Yu, C.-C. Liu, P.-S. Chen, H.-C. Chien, J.-H. Cheng, L.-L. Liao, M.-J. Dai, Y.-M. Lin *et al.*, "Performance and process comparison between glass and si interposer for 3d-ic integration," in *International Symposium on Microelectronics*, vol. 2013, no. 1. International Microelectronics Assembly and Packaging Society, 2013, pp. 000 618–000 624.

[16] J. Choquette, "NVIDIA Hopper H100 GPU: Scaling performance," *IEEE Micro*, vol. 43, no. 3, pp. 9–17, 2023.

[17] J. Choquette, W. Gandhi, O. Giroux, N. Stam, and R. Krashinsky, "NVIDIA A100 Tensor Core GPU: Performance and innovation," *IEEE Micro*, vol. 41, no. 2, pp. 29–35, 2021.

[18] Corning Incorporated. (2026) Advanced packaging glass carriers. Corning Precision Glass Solutions. [Online]. Available: https://www.corning.com/worldwide/en/products/advanced-optics/product-materials/PrecisionGlassSolutions/advanced-packaging-carriers.html

[19] W. J. Dally and B. P. Towles, *Principles and Practices of Interconnection Networks*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2004.

[20] DeepSeek-AI, "Deepseek-v4 technical report," https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf, Apr. 2026, accessed: 2026-05-18.

[21] DeepSeek-AI, others Aixin Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Wang, J. Chen, J. Chen, J. Yuan, J. Qiu, J. Li, J. Song, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Wang, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Wang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Zhang, R. Pan, R. Wang, R. Xu, R. Zhang, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Pan, T. Wang, T. Yun, T. Pei, T. Sun, W. L. Xiao, W. Zeng, W. Zhao, W. An, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Zhang, X. Chen, X. Nie, X. Sun, X. Wang, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Song, X. Shan, X. Zhou, X. Yang, X. Li, X. Su, X. Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Y. Zhang, Y. Xu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Yu, Y. Zheng, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Wu, Y. Ou, Y. Zhu, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Zha, Y. Xiong, Y. Ma, Y. Yan, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. F. Wu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Huang, Z. Zhang, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Xu, Z. Wu, Z. Zhang, Z. Li, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Gao, and Z. Pan, "Deepseek-v3 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2412.19437

[22] N. Ding, Y. Chen, B. Xu, Y. Qin, S. Hu, Z. Liu, M. Sun, and B. Zhou, "Enhancing chat language models by scaling high-quality instructional conversations," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, 2023, pp. 3029–3051.

[23] G. Djogo *et al.*, "Glass interposer for short reach optical connectivity," *Optics Express*, vol. 24, no. 11, p. 12375, 2016.

[24] P. Dong, W. Qian, H. Liang, R. Shafiiha, N.-N. Feng, D. Feng, X. Zheng, A. V. Krishnamoorthy, and M. Asghari, "Low power and compact reconfigurable multiplexing devices based on silicon microring resonators," *Optics Express*, vol. 18, no. 10, pp. 9852–9858, 2010.

[25] A. Y. et al., "Qwen3 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[26] M. Glick, N. C. Abrams, Q. Cheng, M. Y. Teh, Y.-H. Hung, O. Jimenez, S. Liu, Y. Okawachi, X. Meng, L. Johansson, M. Ghobadi, L. Dennison, G. Michelogiannakis, J. Shalf, A. Liu, J. Bowers, A. L. Gaeta, M. Lipson, and K. Bergman, "PINE: Photonic integrated networked energy efficient datacenters (ENLITENED program)," *Journal of Optical Communications and Networking*, vol. 12, no. 12, pp. 443–456, 2020.

[27] J. R. Grenier, L. Brusberg, K. A. Wieland, J. Matthies, and C. C. Terwilliger, "Ultrafast laser processing of glass waveguide substrates for multi-fiber connectivity in co-packaged optics," *Advanced Optical Technologies*, vol. 12, p. 1244009, 2023. [Online]. Available: https://doi.org/10.3389/aot.2023.1244009

[28] J.-H. Han, R. E. West, K. Torres-Castro, N. Swami, S. Khan, and M. Stan, "Power and thermal modeling of in-3d-memory computing," in *2021 International Symposium on Devices, Circuits and Systems (ISDCS)*. IEEE, 2021, pp. 1–4.

[29] C. He, Y. Huang, P. Mu, Z. Miao, J. Xue, L. Ma, F. Yang, and L. Mai, "WaferLLM: Large language model inference at wafer scale."

[30] T.-C. Hsueh, B. Lin, Z. Chen, and Y. Fainman, "Panel-scale reconfigurable photonic interconnects for scalable AI computation," vol. 5, pp. 437–453. [Online]. Available: https://ieeexplore.ieee.org/document/11201050/

[31] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu *et al.*, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," *Advances in neural information processing systems*, vol. 32, 2019.

[32] Huawei. (2026, Mar.) Huawei unveiled the latest SuperPoD, making an AI infrastructure new option to the world. Huawei News. [Online]. Available: https://www.huawei.com/en/news/2026/3/mwc-superpod-ai

[33] Intel Corporation, "Intel unveils industry-leading glass substrates to meet demand for more powerful compute," https://newsroom.intel.com/

artificial-intelligence/intel-unveils-industry-leading-glass-substrates, Sep. 2023, accessed: 2026-05-18.

[34] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de las Casas, E. B. Hanna, F. Bressand, G. Lengyel, G. Bour, G. Lample, L. R. Lavaud, L. Saulnier, M.-A. Lachaux, P. Stock, S. Subramanian, S. Yang, S. Antoniak, T. L. Scao, T. Gervet, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mixtral of experts," 2024. [Online]. Available: https://arxiv.org/abs/2401.04088

[35] M. Khani, M. Ghobadi, M. Alizadeh, Z. Zhu, M. Glick, K. Bergman, A. Vahdat, B. Klenk, and E. Ebrahimi, "SiP-ML: high-bandwidth optical network interconnects for machine learning training," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. ACM, pp. 657–675. [Online]. Available: https://dl.acm.org/doi/10.1145/3452296.3472900

[36] Kimi-Team, Y. Bai, Y. Bao, Y. Charles, C. Chen, G. Chen, H. Chen, H. Chen, J. Chen, N. Chen, R. Chen, Y. Chen, Y. Chen, Y. Chen, Z. Chen, J. Cui, H. Ding, M. Dong, A. Du, C. Du, D. Du, Y. Du, Y. Fan, Y. Feng, K. Fu, B. Gao, C. Gao, H. Gao, P. Gao, T. Gao, Y. Ge, S. Geng, Q. Gu, X. Gu, L. Guan, H. Guo, J. Guo, X. Hao, T. He, W. He, W. He, Y. He, C. Hong, H. Hu, Y. Hu, Z. Hu, W. Huang, Z. Huang, Z. Huang, T. Jiang, Z. Jiang, X. Jin, Y. Kang, G. Lai, C. Li, F. Li, H. Li, M. Li, W. Li, Y. Li, Y. Li, Y. Li, Z. Li, Z. Li, H. Lin, X. Lin, Z. Lin, C. Liu, C. Liu, H. Liu, J. Liu, J. Liu, L. Liu, S. Liu, T. Y. Liu, T. Liu, W. Liu, Y. Liu, Y. Liu, Y. Liu, Y. Liu, Z. Liu, E. Lu, H. Lu, L. Lu, Y. Luo, S. Ma, X. Ma, Y. Ma, S. Mao, J. Mei, X. Men, Y. Miao, S. Pan, Y. Peng, R. Qin, Z. Qin, B. Qu, Z. Shang, L. Shi, S. Shi, F. Song, J. Su, Z. Su, L. Sui, X. Sun, F. Sung, Y. Tai, H. Tang, J. Tao, Q. Teng, C. Tian, C. Wang, D. Wang, F. Wang, H. Wang, H. Wang, J. Wang, J. Wang, J. Wang, S. Wang, S. Wang, S. Wang, X. Wang, Y. Wang, Y. Wang, Y. Wang, Y. Wang, Y. Wang, Z. Wang, Z. Wang, Z. Wang, Z. Wang, C. Wei, Q. Wei, H. Wu, W. Wu, X. Wu, Y. Wu, C. Xiao, J. Xie, X. Xie, W. Xiong, B. Xu, J. Xu, L. H. Xu, L. Xu, S. Xu, W. Xu, X. Xu, Y. Xu, Z. Xu, J. Xu, J. Xu, J. Yan, Y. Yan, H. Yang, X. Yang, Y. Yang, Y. Yang, Z. Yang, Z. Yang, Z. Yang, H. Yao, X. Yao, W. Ye, Z. Ye, B. Yin, L. Yu, E. Yuan, H. Yuan, M. Yuan, S. Yuan, H. Zhan, D. Zhang, H. Zhang, W. Zhang, X. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Z. Zhang, H. Zhao, Y. Zhao, Z. Zhao, H. Zheng, S. Zheng, L. Zhong, J. Zhou, X. Zhou, Z. Zhou, J. Zhu, Z. Zhu, W. Zhuang, and X. Zu, "Kimi k2: Open agentic intelligence," 2026. [Online]. Available: https://arxiv.org/abs/2507.20534

[37] J. Krajewski, J. Ludziejewski, K. Adamczewski, M. Pióro, M. Krutul, S. Antoniak, K. Ciebiera, K. Król, T. Odrzygóźdź, P. Sankowski, M. Cygan, and S. Jaszczur, "Scaling laws for fine-grained mixture of experts," 2024. [Online]. Available: https://arxiv.org/abs/2402.07871

[38] Y. Lei, D. Lee, L. Zhao, D. Kurniawan, C. Kim, H. b. Jeong, C. Kim, H. Choi, L. Yu, A. Krishnamurthy, J. Sherry, and E. Nurvitadhi, "FAST: An efficient scheduler for All-to-All GPU communication," in *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, 2026, pp. 2515–2531.

[39] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "Gshard: Scaling giant models with conditional computation and automatic sharding," *arXiv preprint arXiv:2006.16668*, 2020.

[40] D. Li, E. Zhang, Y. Xu, L. Zhang, Y. Zhang, and X. Yang, "Low-loss silicon nitride bent waveguides at O-band with modified hermite curves," *Photonics*, vol. 13, no. 2, p. 175, 2026.

[41] X. Li and M. Hamdi, "On scheduling optical packet switches with reconfiguration delay," *IEEE Journal on Selected Areas in Communications*, vol. 21, no. 7, pp. 1156–1164, 2003.

[42] K. Liang, L. Qiao, I. Keslassy, and B. Lin, "Scheduling parallel optical switches in datacenters," *IEEE Networking Letters*, 2026.

[43] X. Liao, Y. Sun, H. Tian, X. Wan, Y. Jin, Z. Wang, Z. Ren, X. Huang, W. Li, K. F. Tse, Z. Zhong, G. Liu, Y. Zhang, X. Ye, Y. Zhang, and K. Chen, "MixNet: A runtime reconfigurable optical-electrical fabric for distributed mixture-of-experts training," in *Proceedings of the ACM SIGCOMM 2025 Conference*. ACM, pp. 554–574. [Online]. Available: https://dl.acm.org/doi/10.1145/3718958.3750465

[44] S. Lie, "The cerebras wafer-scale architecture for deep learning," Cerebras Systems, White Paper, author listed as Co-Founder and CTO, Cerebras Systems. [Online]. Available: https://cerebras.ai

[45] M.-S. Lin, C.-C. Tsai, S. Li, W.-C. Chen, W.-H. Huang, Y.-C. Chen, Y.-J. Huang, A. Drake, C.-H. Wen, P. Ranucci, H.-H. Kuo, A. Yin, S.-C. Yang, F. Mahmoudi, H.-T. Ke, C.-C. Li, N.-C. Cheng, J. Wang, K. Lin, H. Liao, J.-R. Huang, M.-H. Wu, K. C.-H. Hsieh, N. Amatruda, W. Polanco, D. King, T. Basso, and A. Kashem, "36.1 a 32gb/s 10.5tb/s/mm 0.6pj/b ucie-compliant low-latency interface in 3nm featuring matched-delay for dynamic clock gating," in *2025 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 68, 2025, pp. 586–588.

[46] Y.-C. Lin, W. Kwon, R. Pineda, and F. N. Paravecino, "Apex: An extensible and dynamism-aware simulator for automated parallel execution in llm serving," 2024. [Online]. Available: https://arxiv.org/abs/2411.17651

[47] D. Liu, Z. Yan, X. Yao, T. Liu, V. Korthikanti, E. Wu, S. Fan, G. Deng, H. Bai, J. Chang *et al.*, "Moe parallel folding: Heterogeneous parallelism mappings for efficient large-scale moe model training with megatron core," *arXiv preprint arXiv:2504.14960*, 2025.

[48] P. Marin-Palomo, J. N. Kemal, M. Karpov, A. Kordts, J. Pfeifle, M. H. P. Pfeiffer, P. Trocha, S. Wolf, V. Brasch, M. H. Anderson, R. Rosenberger, K. Vijayan, W. Freude, T. J. Kippenberg, and C. Koos, "Microresonator-based solitons for massively parallel coherent optical communications," *Nature*, vol. 546, pp. 274–279, 2017.

[49] P. McLellan, "Whatever happened to 450 mm wafers?" Cadence Breakfast Bytes Blog, Aug. 2022, accessed May 2026. [Online]. Available: https://community.cadence.com/cadence_blogs_8/b/breakfastbytes/posts/450mm-wafers

[50] G. Michelogiannakis, Y. Arafa, B. Cook, L. Y. Dai, A.-H. A. Badawy, M. Glick, Y. Wang, K. Bergman, and J. Shalf, "Efficient intra-rack resource disaggregation for HPC using co-packaged DWDM photonics," in *2023 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 2023, pp. 158–172.

[51] G. Michelogiannakis, Y. Shen, M. Y. Teh, X. Meng, B. Aivazi, T. Groves, J. Shalf, M. Glick, M. Ghobadi, L. Dennison, and K. Bergman, "Bandwidth steering in HPC using silicon nanophotonics," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '19. ACM, 2019.

[52] Micron Technology, Inc., "Micron HBM3E Product Brief," https://assets.micron.com/adobe/assets/urn:aaid:aem:b710d8f2-7f66-44c1-a234-456e2b986347/renditions/original/as/hbm3e-product-brief.pdf, Oct. 2023, rev. C, October 2023. Accessed: 2026-07-28.

[53] D. A. B. Miller, "Device requirements for optical interconnects to silicon chips," *Proceedings of the IEEE*, vol. 97, no. 7, pp. 1166–1185, 2009.

[54] D. A. B. Miller and H. M. Ozaktas, "Limit to the bit-rate capacity of electrical interconnects from the aspect ratio of the system architecture," *Journal of Parallel and Distributed Computing*, vol. 41, no. 1, pp. 42–52, 1997.

[55] C. Minkenberg, R. Krishnaswamy, A. Zilkie, and D. Nelson, "Co-packaged datacenter optics: Opportunities and challenges," *IET Optoelectronics*, vol. 15, no. 2, pp. 77–91, 2021.

[56] Y. Mizuno, K. Kobayashi, S. Nakamura, M. Migita, H. Arao, T. Nakanishi, H. Uemura, K. Tanaka, K. Uesaka, M. Miyairi, Y. Ishizuki, and Y. Kurita, "Flip-chip photonic-electronic integration platform for co-packaged optics using a glass substrate with vertically-coupled beam expanding lens," in *2025 IEEE 75th Electronic Components and Technology Conference (ECTC)*, 2025, pp. 48–53.

[57] N. Muennighoff, L. Soldaini, D. Groeneveld, K. Lo, J. Morrison, S. Min, W. Shi, P. Walsh, O. Tafjord, N. Lambert, Y. Gu, S. Arora, A. Bhagia, D. Schwenk, D. Wadden, A. Wettig, B. Hui, T. Dettmers, D. Kiela, A. Farhadi, N. A. Smith, P. W. Koh, A. Singh, and H. Hajishirzi, "Olmoe: Open mixture-of-experts language models," 2024. [Online]. Available: https://arxiv.org/abs/2409.02060

[58] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. A. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro *et al.*, "Efficient large-scale language model training on gpu clusters using megatron-lm," in *SC '21: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021.

[59] H. Nejadriahi, S. Pappert, Y. Fainman, and P. Yu, "Efficient and compact thermo-optic phase shifter in silicon-rich silicon nitride," *Optics Letters*, vol. 46, no. 18, 2021, arXiv:2106.09476.

[60] P. Nimbalkar, P. Bhaskar, L. N. Vijay Kumar, M. Narayanan, E. Torres, S. S. Ambi Venkataramanan, and M. Kathaperumal, "A review of glass substrate technologies," *Chips*, vol. 4, no. 3, 2025. [Online]. Available: https://www.mdpi.com/2674-0729/4/3/37

[61] NVIDIA Corporation, "cuBLAS: Basic linear algebra subprograms," https://developer.nvidia.com/cublas, version 12.x/13.x.

[62] NVIDIA Corporation, "NCCL Tests," https://github.com/NVIDIA/nccl-tests, accessed: 2026-05-27.

[63] NVIDIA Corporation, "System Hardware & Components," *NVIDIA NVL72 AI Factory*, 2026, last updated May 18, 2026. Accessed May 24, 2026. [Online]. Available: https://docs.nvidia.com/enterprise-reference-architectures/nvl72-ai-factory/latest/components.html

[64] OpenAI, "ChatGPT," https://chat.openai.com/, 2026, large language model.

[65] Optical Internetworking Forum, "Co-packaging framework document," Optical Internetworking Forum, Tech. Rep. OIF-Co-Packaging-FD-01.0, Feb. 2022. [Online]. Available: https://www.oiforum.com/wp-content/uploads/OIF-Co-Packaging-FD-01.0.pdf

[66] Optical Internetworking Forum, "Common electrical I/O (CEI)-112g implementation agreement," https://www.oiforum.com/technical-work/hot-topics/common-electrical-interface-cei-112g-2/, 2022, standard.

[67] G. Penedo, H. Kydlíček, L. Ben Allal, A. Lozhkov, M. Mitchell, C. A. Raffel, L. von Werra, and T. Wolf, "The FineWeb datasets: Decanting the web for the finest text data at scale," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 30 811–30 849.

[68] J. W. Rae, A. Potapenko, S. M. Jayakumar, C. Hillier, and T. P. Lillicrap, "Compressive transformers for long-range sequence modelling," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=SylKikSYDH

[69] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale."

[70] S. Rashidi, W. Won, S. Srinivasan, P. Gupta, and T. Krishna, "FRED: A wafer-scale fabric for 3d parallel DNN training," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. ACM, pp. 34–48. [Online]. Available: https://dl.acm.org/doi/10.1145/3695053.3731055

[71] A. Reitz, A. Evertz, D. Basten, M. C. Wurz, and L. Overmeyer, "Integrated multimode optical waveguides in glass using laser induced deep etching," *Applied Optics*, vol. 63, no. 4, pp. 895–903, 2024.

[72] S. Renganathan and N. McKeown, "Chronos: Prescheduled circuit switching for LLM training," in *Proceedings of the 2nd Workshop on Networks for AI Computing (NAIC)*. Coimbra, Portugal: ACM, 2025, pp. 89–97.

[73] H. Schröder, N. Arndt-Staufenbiel, M. Cygon, and W. Scheel, "Planar glass waveguides for high performance electrical-optical-circuit-boards (eocb): The glass-layer-concept," in *53rd Electronic Components and Technology Conference (ECTC)*, 2003, pp. 1053–1059.

[74] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer." [Online]. Available: http://arxiv.org/abs/1701.06538

[75] P.-C. Shih, A.-J. Su, K.-H. Tam, T.-C. Huang, K. Chuang, and J. Yeh, "Sow-x: A novel system-on-wafer technology for next generation ai server application," in *2025 IEEE 75th Electronic Components and Technology Conference (ECTC)*. IEEE, 2025, pp. 1–6.

[76] C. Sun, C.-H. O. Chen, G. Kurian, L. Wei, J. Miller, A. Agarwal, L.-S. Peh, and V. Stojanovic, "DSENT - a tool connecting emerging photonics with electronics for opto-electronic networks-on-chip modeling," in *2012 IEEE/ACM Sixth International Symposium on Networks-on-Chip*. IEEE, pp. 201–210. [Online]. Available: http://ieeexplore.ieee.org/document/6209280/

[77] C. Sun, M. T. Wade, Y. Lee, J. S. Orcutt, L. Alloatti, M. S. Georgas, A. S. Waterman, J. M. Shainline, R. R. Avizienis, S. Lin, B. R. Moss, R. Kumar, F. Pavanello, A. H. Atabaki, H. M. Cook, A. J. Ou, J. C. Leu, Y.-H. Chen, K. Asanović, R. J. Ram, M. A. Popović, and V. M. Stojanović, "Single-chip microprocessor that communicates directly using light," *Nature*, vol. 528, pp. 534–538, 2015.

[78] E. Talpes, D. D. Sarma, D. Williams, S. Arora, T. Kunjan, B. Floering, A. Jalote, C. Hsiong, C. Poorna, V. Samant, J. Sicilia, A. K. Nivarti, R. Ramachandran, T. Fischer, B. Herzberg, B. McGee, G. Venkataramanan, and P. Banon, "The microarchitecture of dojo, tesla's exa-scale computer," *IEEE Micro*, vol. 43, no. 3, p. 31–39, May 2023. [Online]. Available: https://doi.org/10.1109/MM.2023.3258906

[79] X. Tang, J. Hou, D. Jiang, T. Wei, J. Liu, J. Deng, H. Wang, Q. Yang, H. Shang, C. Li *et al.*, "Moentwine: Unleashing the potential of wafer-scale chips for large-scale expert parallel inference," in *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2026, pp. 1–15.

[80] K. Team, T. Bai, Y. Bai, Y. Bao, M. C., J. Cai, X. Cai, P. Cao, Y. Cao, Z. Chai, Y. Charles, H. S. Che, G. Chen, G. Chen, G. Chen, H. Chen, J. Chen, J. Chen, J. Chen, K. Chen, P. Chen, R. Chen, W. Chen, X. Chen, Y. Chen, Y. Chen, Y. Chen, Y. Chen, Y. Chen, Y. Chen, Y. Chen, Z. Chen, D. Cheng, Y. Cheng, J. Cui, J. Cui, A. Dai, J. Deng, H. Ding, R. Ding, S. Ding, M. Dong, M. Dong, Y. Dong, Y. Dong, A. Du, C. Du, D. Du, J. Du, Y. Du, Y. Fan, J. Feng, Q. Feng, Y. Feng, K. Fu, Q. Fu, F. Gao, H. Gao, J. Gao, T. Gao, W. Gao, S. Geng, J. Gong, L. Gong, S. Gong, X. Gong, Q. Gu, Y. Gu, S. Guan, H. Guo, S. Guo, X. Guo, Z. Guo, B. Hao, W. Hao, X. Hao, D. He, H. He, L. He, Q. He, W. He, X. He, X. He, Y. He, Y. He, C. Hong, T. Hong, H. Hu, J. Hu, R. Hu, W. Hu, Y. Hu, Z. Hu, L. Hua, J. Huang, K. Huang, R. Huang, S. Huang, W. Huang, Y. Huang, Z. Huang, Z. Huang, Y. Hui, C. Jia, Y. Jiang, Z. Jiang, Z. Jiang, W. Jin, X. Jin, Y. Jing, H. Kong, G. Lai, A. Li, C. Li, C. Li, C. Li, F. Li, G. Li, H. Li, J. Li, J. Li, L. Li, L. Li, L. Li, W. Li, W. Li, X. Li, Y. Li, Y. Li, Y. Li, Y. Li, Z. Li, Z. Li, Z. Li, Z. Li, Z. Li, J. Lin, X. Lin, Y. Lin, Z. Lin, Z. Lin, B. Liu, B. Liu, C. Liu, L. Liu, S. Liu, S. Liu, S. Liu, T. Liu, W. Liu, Y. Liu, Y. Liu, Y. Liu, Z. Liu, Z. Liu, E. Lu, H. Lu, L. Lu, T. Lu, Z. Lu, A. Luo, G. Luo, J. Luo, Y. Luo, B. Lyu, W. Lyu, S. Mao, Y. Mei, X. Men, M. Ni, Y. Niu, S. Pan, S. Peng, Z. Qi, R. Qin, Z. Qin, Z. Qin, H. Qiu, J. Qiu, J. Qiu, B. Qu, Y. Qu, Z. Shang, Y. Shao, H. Shen, J. Shi, J. Shi, L. Shi, S. Shi, W. Siu, P. Song, X. Song, J. Su, Y. Su, Z. Su, L. Sui, J. Sun, J. Sun, S. Sun, S. Sun, T. Sun, Y. Sun, Y. Tai, C. Tang, H. Tang, S. Tang, Z. Tang, C. Tian, R. Tian, Y. Tian, W. Tu, C. Wang, C. Wang, C. Wang, D. Wang, F. Wang, H. Wang, H. Wang, H. Wang, H. Wang, H. Wang, H. Wang, J. Wang, J. Wang, J. Wang, J. Wang, L. Wang, S. Wang, S. Wang, S. Wang, S. Wang, S. Wang, T. Wang, W. Wang, X. Wang, X. Wang, X. Wang, X. Wang, Y. Wang, Y. Wang, Y. Wang, Y. Wang, Y. Wang, Y. Wang, Y. Wang, Y. Wang, Z. Wang, Z. Wang, Z. Wang, Z. Wang, Z. Wang, C. Wei, M. Wei, S. Wei, Z. Wen, F. Wu, H. Wu, R. Wu, W. Wu, X. Wu, Y. Wu, Y. Wu, Y. Wu, Z. Wu, X. Xian, C. Xiang, Y. Xiang, B. Xiao, C. Xiao, X. Xiao, J. Xie, X. Xie, Y. Xie, Z. Xie, B. Xing, Y. Xiong, B. Xu, B. Xu, J. Xu, J. Xu, J. Xu, J. Xu, L. H. Xu, Q. Xu, S. Xu, S. Xu, T. Xu, T. Xu, W. Xu, X. Xu, Y. Xu, Y. Xu, Y. Xu, Z. Xu, H. Xue, J. Yan, Y. Yan, F. Yang, G. Yang, H. Yang, J. Yang, R. Yang, W. Yang, X. Yang, X. Yang, Y. Yang, Y. Yang, Y. Yang, Y. Yang, Z. Yang, Z. Yang, Z. Yang, Z. Yang, H. Yao, D. Ye, H. Ye, W. Ye, Z. Ye, B. Yin, H. Yin, X. Yin, C. Yu, H. Yu, L. Yu, S. Yu, S. Yu, T. Yu, E. Yuan, M. Yuan, T. Yue, W. Yue, Y. Yue, D. Zha, H. Zhan, B. H. Zhang, D. Zhang, F. Zhang, H. Zhang, H. Zhang, H. Zhang, J. Zhang, J. Zhang, J. Zhang, K. Zhang, M. Zhang, P. Zhang, Q. Zhang, R. Zhang, R. Zhang, S. Zhang, S. Zhang, X. Zhang, X. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, Z. Zhang, Z. Zhang, B. Zhao, C. Zhao, F. Zhao, J. Zhao, J. Zhao, S. Zhao, W. Zhao, X. Zhao, X. Zhao, Y. Zhao, Z. Zhao, H. Zheng, H. Zheng, R. Zheng, S. Zheng, T. Zheng, H. Zhong, L. Zhong, L. Zhong, M. Zhou, Q. Zhou, R. Zhou, R. Zhou, X. Zhou, Y. Zhou, Z. Zhou, J. Zhu, L. Zhu, X. Zhu, Y. Zhu, Y. Zhu, Z. Zhu, C. Zhuang, W. Zhuang, and X. Zu, "Kimi k3: Open frontier intelligence," 2026. [Online]. Available: https://arxiv.org/abs/2607.24653

[81] F. Terraneo, A. Leva, W. Fornaciari, M. Zapater, and D. Atienza, "3D-ICE 3.0: Efficient nonlinear mpsoc thermal simulation with pluggable heat sink models," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 4, pp. 1062–1075, 2022.

[82] A. Usman, E. Shah, N. B. Satishprasad, J. Chen, S. A. Bohlemann, S. H. Shami, A. A. Eftekhar, and A. Adibi, "Interposer technologies for high-performance applications," *IEEE Transactions on Components, Packaging and Manufacturing Technology*, vol. 7, no. 6, pp. 819–832, 2017.

[83] P. Vanna-Iampikul, S. Woo, S. Erdogan, L. Zhu, M. Kathaperumal, R. Agarwal, R. Gupta, K. Rinebold, M. Swaminathan, and S. K. Lim, "Glass interposer integration of logic and memory chiplets: Ppa and power/signal integrity benefits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 5, pp. 1954–1967, 2024.

[84] D. Vantrease, R. Schreiber, M. Monchiero, M. McLaren, N. P. Jouppi, M. Fiorentino, A. Davis, N. Binkert, R. G. Beausoleil, and J. H. Ahn, "Corona: System implications of emerging nanophotonic technology," *SIGARCH Comput. Archit. News*, vol. 36, no. 3, p. 153–164, Jun. 2008. [Online]. Available: https://doi.org/10.1145/1394608.1382135

[85] J. von Neumann, "A certain zero-sum two-person game equivalent to the optimal assignment problem," in *Contributions to the Theory of Games (AM-28), Volume II*. Princeton University Press, 1953, pp. 5–12.

[86] W. Wang, M. Ghobadi, K. Shakeri, Y. Zhang, and N. Hasani, "Rail-only:

A low-cost high-performance network for training LLMs with trillion parameters," in *IEEE Symposium on High-Performance Interconnects (HOTI)*, 2024.

[87] Z. Wu, Y. Wang, and K. Bergman, "Sipam: Silicon photonic accelerated memory pooling architecture," *IEEE Micro*, 2026.

[88] Z. Xu, D. Kong, J. Liu, J. Li, J. Hou, X. Dai, C. Li, S. Wei, Y. Hu, and S. Yin, "WSC-LLM: Efficient LLM service and architecture co-exploration for wafer-scale chips," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. ACM, pp. 1–17. [Online]. Available: https://dl.acm.org/doi/10.1145/3695053.3731101

[89] X. Yu, D. Jiang, J. Deng, J. Liu, C. Li, S. Yin, and Y. Hu, "Cramming a data center into one cabinet, a co-exploration of computing and hardware architecture of waferscale chip," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 631–645.

[90] T. Zadouri, M. Hoehnerbach, J. Shah, T. Liu, V. Thakkar, and T. Dao, "Flashattention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling," 2026. [Online]. Available: https://arxiv.org/abs/2603.05451

[91] J. Zhu, C. Xue, Y. Chen, Z. Wang, C. Zhang, Y. Shen, Y. Chen, Z. Cheng, J. Yu, T. Wang, Y. Lin, W. Hu, B. Cui, R. Wang, Y. Liang, and G. Sun, "Theseus: Exploring efficient wafer-scale chip design for large language models," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 12, pp. 4793–4806, 2025.